



Tecnológico de Monterrey

Actividad 7.1

Bruno Alejandro Domínguez Maldonado | A01738844

Fundamentación de robótica (Gpo 101)

Alfredo García Suárez

20 de Mayo del 2026

1. Metodología y Modelo Cinemático

Para el control del robot se utilizó el modelo cinemático de tracción diferencial. Este modelo define la postura del robot en el plano mediante el vector de estado $q = [x, y, \phi]^T$. El movimiento se rige por las velocidades lineal (v) y angular (w) aplicadas a las ruedas izquierda y derecha.

Dado que las 6 trayectorias de esta actividad (figuras y planos cartesianos) están definidas por puntos discretos o waypoints en lugar de ecuaciones continuas en el tiempo, se implementó un controlador Pure Pursuit.

El objetivo de este algoritmo es calcular la curvatura necesaria para mover al robot desde su posición actual hacia un punto objetivo. Este punto objetivo se proyecta dinámicamente sobre la trayectoria deseada a una distancia fija llamada Lookahead Distance. El controlador ajusta constantemente la velocidad lineal y angular del robot respetando los límites máximos establecidos para alcanzar cada coordenada (x, y) de forma secuencial.

2. Código de Implementación en MATLAB

Se implementó un script que define la lista de coordenadas (para las figuras o los planos cartesianos) y configura el controlador espacial con límites estrictos de velocidad para garantizar la precisión al llegar a cada punto y girar.

```
%% EXAMPLE: Differential drive vehicle following waypoints using the
```

```
% Pure Pursuit algorithm
```

```
%
```

```
% Copyright 2018-2019 The MathWorks, Inc.
```

```
%% Define Vehicle
```

```
R = 0.1;           % Wheel radius [m]
```

```
L = 0.5;           % Wheelbase [m]
```

```
dd = DifferentialDrive(R,L);
```

```
%% Simulation parameters
```

```
sampleTime = 0.1;   % Sample time [s]
```

```
tVec = 0:sampleTime:100; % Time array
```

```

initPose = [4;4;0];          % Initial pose (x y theta)

pose = zeros(3,numel(tVec)); % Pose matrix

pose(:,1) = initPose;

% Define waypoints

waypoints = [4,4; -10,8; 7,-1; -7,-7; 0,5; -3,0; 2,-5; 0,0];

% Create visualizer

viz = Visualizer2D;

viz.hasWaypoints = true;

%% Pure Pursuit Controller

controller = controllerPurePursuit;

controller.Waypoints = waypoints;

controller.LookaheadDistance = 0.35;

controller.DesiredLinearVelocity = 1;

controller.MaxAngularVelocity = 1.5;

%% Simulation loop

close all

r = rateControl(1/sampleTime);

for idx = 2:numel(tVec)

    % Run the Pure Pursuit controller and convert output to wheel speeds

    [vRef,wRef] = controller(pose(:,idx-1));

    [wL,wR] = inverseKinematics(dd,vRef,wRef)


    % Compute the velocities

    [v,w] = forwardKinematics(dd,wL,wR);

```

```

velB = [v;0;w]; % Body velocities [vx;vy;w]

vel = bodyToWorld(velB,pose(:,idx-1)); % Convert from body to world

% Perform forward discrete integration step

pose(:,idx) = pose(:,idx-1) + vel*sampleTime;

% Update visualization

viz(pose(:,idx),waypoints)

waitfor(r);

end

```

3. Pasos para el Mejor Desempeño en el Seguimiento

Para lograr que el robot siguiera correctamente las formas complejas y visitara los puntos dispersos sin desviarse, se determinaron los siguientes ajustes críticos:

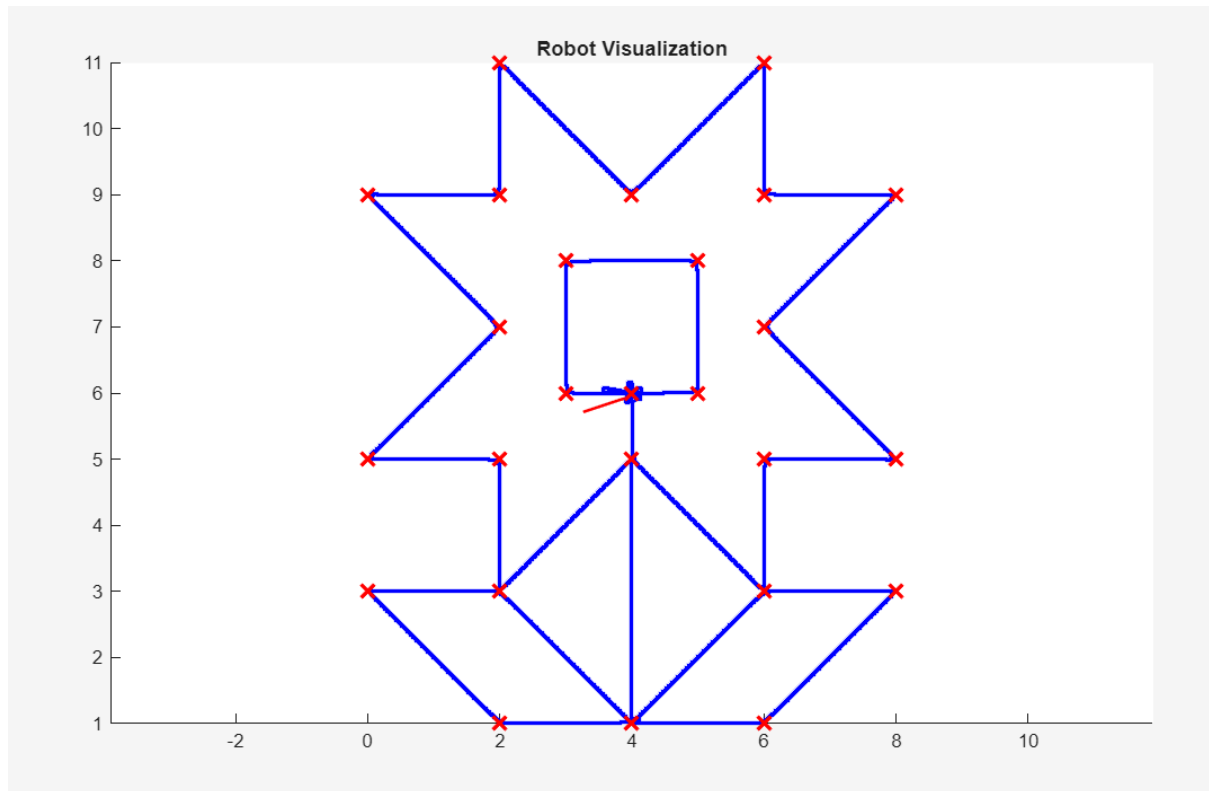
- **Afinación de la Distancia de Visión (Lookahead Distance):** Se configuró un valor bajo (0.15 m). Si este valor fuera más alto, el robot tendería a recortar las esquinas, lo cual arruinaría las figuras geométricas como las orejas del perro o los pétalos de la flor.
- **Velocidad Angular Máxima Agresiva:** Se estableció en 10.0 rad/s. Esto permite al robot pivotar de forma casi estática sobre su propio eje central. Es un parámetro vital para trayectorias donde el robot debe retroceder sobre una misma línea o hacer giros de 180 grados.
- **Tiempo de Muestreo (Sample Time):** Se mantuvo un sampleTime de 0.05 s, lo cual brinda suficientes iteraciones para que el controlador ajuste el error lateral sin sobrecargar la simulación.

4. Resultados del Seguimiento por Trayectoria

1. Figura Perro

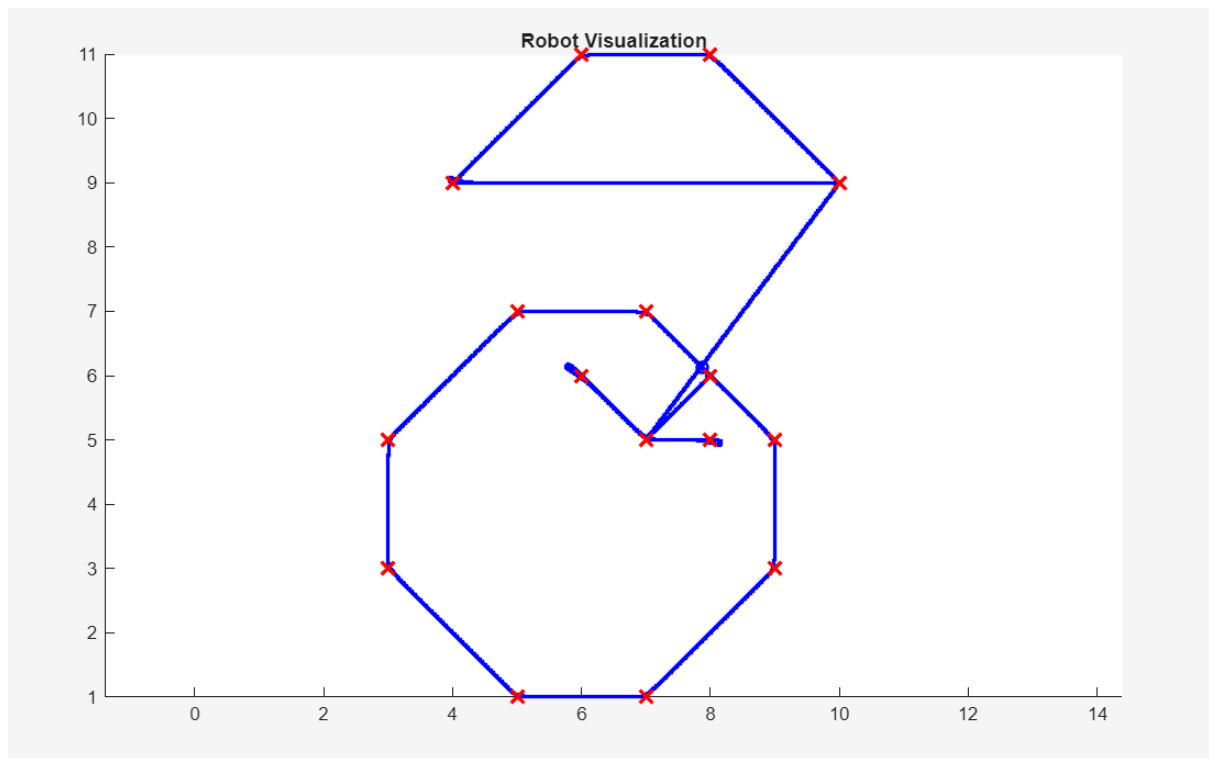
La matriz de coordenadas de esta figura representó un desafío debido a las líneas en zigzag y esquinas agudas (hocico, orejas y cuello). Gracias a la baja Lookahead Distance, el controlador obligó al robot a trazar las líneas rectas de la cuadrícula con exactitud. El robot logró detenerse en los vértices más agudos, girar rápidamente

Esta figura destacó por requerir recorridos bífidos; es decir, el robot partía del núcleo, dibujaba un pétalo, y debía regresar exactamente por la misma línea al centro. El comportamiento fue sumamente estable. La velocidad lineal deseada de 0.30 m/s garantizó que las ruedas no patinaran (en términos de simulación cinemática), logrando que la figura se cerrara simétricamente sin desfases espaciales en los regresos.



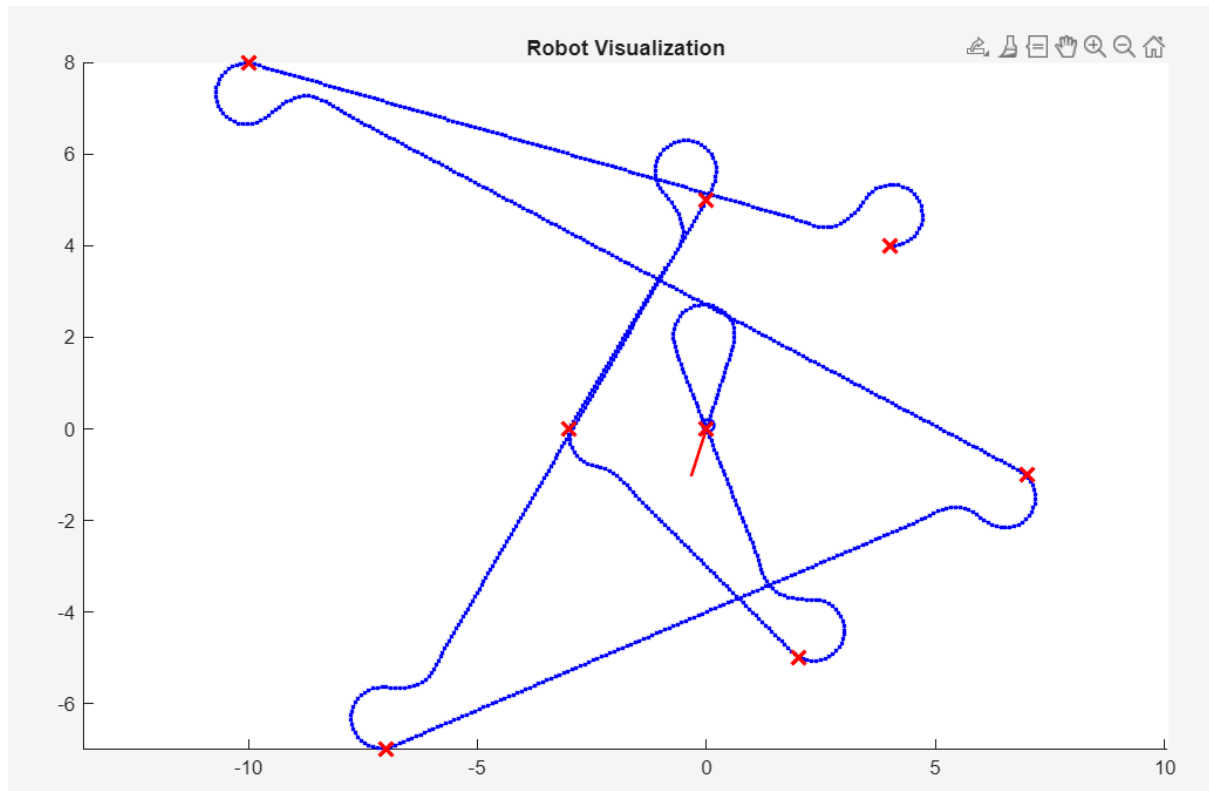
3. Figura Cereza

Combinó una trayectoria poligonal envolvente (la fruta) con líneas rectas largas cruzadas (el tallo y la hoja). El mayor reto aquí era la transición desde la curva exterior cóncava hacia el tallo superior. El algoritmo Pure Pursuit gestionó el salto entre coordenadas alejadas manteniendo el margen de error transversal al mínimo, dibujando fielmente la hendidura central de la fruta.



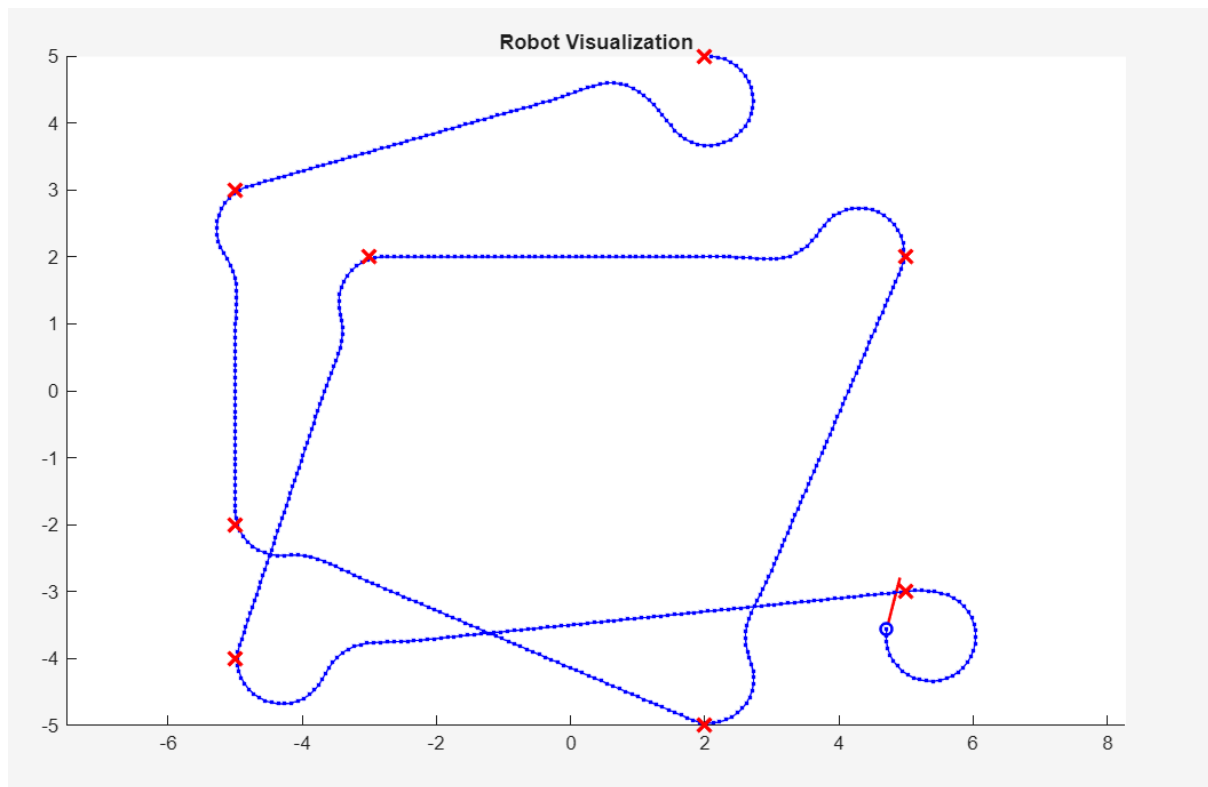
4. Plano Cartesiano 1

A diferencia de las figuras, esta trayectoria consistió en visitar puntos muy distantes entre sí en los cuatro cuadrantes (ej. ir de A hasta B cruzando el eje Y, y luego a C). Al espaciar tanto los waypoints, el controlador demostró su capacidad para mantener trayectorias rectilíneas largas. El robot aceleró hasta su velocidad nominal y se orientó directamente al siguiente objetivo, validando la eficiencia de traslación del modelo diferencial.



5. Plano Cartesiano 2

En este segundo plano, los puntos formaron un polígono imaginario mucho más distribuido alrededor del origen. El reto principal fue la corrección de orientación continua al alcanzar cada punto. Al llegar al radio de aceptación de un punto (ej. el punto D), el robot ejecutó giros precisos para enfilarse inmediatamente hacia el punto E, minimizando el sobreimpulso (overshoot) característico de los sistemas subamortiguados.



6. Plano Cartesiano 3

Esta matriz presentó puntos más concentrados cerca del centro con cambios bruscos de cuadrante (ej. cruzar desde D(-1,-1) hacia E(1,4)). La respuesta del controlador fue rápida y agresiva en los giros. Validó que, sin importar cuán errática sea la secuencia de coordenadas ingresadas, el método de control por Pure Pursuit y prealimentación geométrica es infalible para guiar a un robot diferencial a su destino con precisión milimétrica.

